

Technical Deployment Guide

Audience: a system administrator comfortable with git, Node.js, PostgreSQL, and Windows Server administration, who has **not** worked with this codebase before. This document takes you from `git clone` to a running production server **and** covers what needs to happen on the individual workstations (POS terminals, printer/scale machines, remote sites) that connect to it.

Status: the repo's deployment documentation was consolidated down to two co-canonical files, each verified against the actual current code and scripts rather than carried forward from older prose. This file covers architecture, the full narrative walkthrough, workstation/agent/printer deep-dives, and troubleshooting playbooks. `ADMIN-INSTALLATION-GUIDE.md` covers the same install in step-by-step numbered-checklist form, plus SSL certificate setup and client-machine trust — the two cross-reference each other throughout and neither supersedes the other. The various older, single-purpose deployment/setup/fresh-install notes that used to sit at the repo root have been deleted (fully superseded, some actively wrong — e.g. a stale `.env` vs `.env.local` instruction) or archived to `ai-contexts/project-plans/completed/` (historical, ticket-specific incident writeups, not general install instructions).

1. Architecture in one paragraph

This is a single Next.js 15 application with a **custom Node HTTP server** (`server.ts`, not `next start`) that also hosts a Socket.io server (used for real-time sync, the customer display, and the R710/Workstation local-agent protocols), a PostgreSQL database via Prisma, and an optional standalone Windows Service wrapper that keeps the whole thing running and can rebuild itself. There is no reverse proxy, load balancer, or container setup checked into the repo — the app is designed to be reached directly on its own port. Workstations are plain thin clients — a browser, and optionally a small local helper program (QZ Tray, the R710/Workstation agent, or an Electron kiosk shell) — nothing workstation-side runs its own copy of the application or database.

2. Prerequisites

Install these on the machine that will run the server, before touching the repo.

Software	Version	Notes
Node.js	20.x LTS	No <code>engines</code> field or <code>.nvmrc</code> is committed. <code>package.json</code> pins <code>next@15.5.4</code> , which requires Node ≥ 18.18 — Node 20 LTS is the safe, actively-supported choice. Ignore the "Node.js 16+" message printed by the installer script (<code>scripts/install/install.js</code>) — that check is stale against the actual Next.js requirement.
npm	ships with Node	No separate version pin found anywhere.
PostgreSQL	18.x	Required specifically if you'll use the automated Windows Service installer (§8) — it registers the app's service with a hard-coded Windows Service Control Manager dependency on a service literally named <code>postgresql-x64-18</code> (<code>windows-service/config.js</code>). If you install a different PostgreSQL major version, either install PG 18 as well or edit that <code>dependencies</code> array before running the service installer, or the service will fail to start. If you're not using the Windows Service installer, any reasonably current PostgreSQL (14+) is fine for the app itself.
Git	any current version	Used to clone the repo; also read at runtime by the Windows Service wrapper to detect when a rebuild is needed (it compares the currently checked-out commit to the commit the last build was made from).
openssl	any current version	Needed to generate the QZ Tray signing certificate (§7.2) — ships with Git for Windows, so usually already present.
mkcert	latest	Only needed if you'll serve the app over HTTPS (§7.1) — required in practice for QZ Tray printing to work in Chrome/Edge.

Native Node modules — platform build tools

Two dependencies compile native code: `bcrypt` (password hashing) and `canvas` (image/label rendering, barcode work). Both ship prebuilt binaries for common platform/Node-ABI combinations, so a plain `npm install` usually just works. If it doesn't (a `node-gyp rebuild` failure during `npm install`):

- Windows:** install the "Desktop development with C++" workload via Visual Studio Build Tools, or run `npm install --global windows-build-tools` (older npm) / ensure Python 3 + the MSVC toolchain is on PATH.

- **Linux:** `sudo apt-get install -y build-essential libcairo2-dev libpango1.0-dev libjpeg-dev libgif-dev librsvg2-dev` (Debian/Ubuntu; adjust package manager for other distros) — these are `canvas`'s standard system dependencies, and are **not documented anywhere else in this repo**.

Is production Windows-only?

In practice, yes, even though the codebase isn't strictly Windows-only:

- `scripts/install/install-service.js` (the `sync` service installer, §8.1) does branch on `os.platform()` and will write a real systemd unit file on Linux.
- But the actual **self-healing production launcher** for the main app server (`windows-service/`, §8.2) uses the `node-windows` package and Windows Service Control Manager APIs — there is no Linux equivalent of it. On Linux you'd be running `node dist/server.js` directly under a plain systemd unit with no auto-rebuild-on-stale-commit behavior.

Everything in this guide works on Linux for running the app itself; §8.2 specifically (the hybrid auto-updating service) is Windows-only.

3. Get the code

```
git clone <your-repo-url> multi-business-multi-apps
cd multi-business-multi-apps
```

Pick the branch/tag you intend to deploy before continuing (`git checkout <branch>`).

4. Environment configuration

The repo ships a `.env.example` at the root — **it is not complete**. Copy it and then add the variables called out below that are missing from it entirely.

```
cp .env.example .env.local
```

4.1 Why `.env.local` and not `.env`

`server.ts` never calls `dotenv` itself — env loading happens implicitly the moment it constructs the Next.js app (`next({ dev, hostname, port })`), which uses Next's own built-in loader. Next's loader reads `.env.local` (and `.env`, `.env.production`, etc.) automatically, so **the running app process is fine with `.env.local`**.

The **Prisma CLI is not the app process** — run directly (`npx prisma migrate deploy`, `npx prisma studio`, ...), it only reads `.env` by default, never `.env.local`. This repo's convention is to keep real credentials in `.env.local` only, so raw `npx prisma ...` commands will fail to find `DATABASE_URL` unless you export it first. Two ways to deal with this:

- Use the project's own wrapper: `npm run db:deploy` (see `package.json`) — it loads `.env.local` via `dotenv` and then runs `prisma migrate deploy` with that environment, so you never hit this problem.
- Or export it manually in your shell before any raw `npx prisma ...` call:

```
# bash
export DATABASE_URL='postgresql://user:pass@localhost:5432/multi_business_db'
```

```
# PowerShell
$env:DATABASE_URL = 'postgresql://user:pass@localhost:5432/multi_business_db'
```

4.2 Required variables

These are enforced or required by code — the app will error or misbehave without them. **None of the three marked ⚠ below are present in `.env.example` at all** — you must add them yourself.

Variable	Required	Purpose	How to generate
DATABASE_URL	Yes	PostgreSQL connection string, read by Prisma	<code>postgresql://<user>:<password>@<host>:5432/<database></code>
NEXTAUTH_SECRET	Yes	Session/token signing secret for NextAuth (<code>src/lib/auth.ts</code>)	<code>node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"</code>
ENCRYPTION_KEY ⚠️	Yes	AES-256 key used to encrypt sensitive stored fields (e.g. device admin passwords) — <code>src/lib/encryption.ts</code> throws on first use if this is unset or not exactly 64 hex characters (32 bytes)	<code>node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"</code>
QZ_PRIVATE_KEY / QZ_CERTIFICATE ⚠️	Only if using QZ Tray printing	Signing keypair for QZ Tray print requests — see the full walkthrough in §7.2	<code>npm run qz:generate-cert</code> (requires <code>openssl</code> on PATH) — writes both values for you

A missing `ENCRYPTION_KEY` is a real, previously-hit failure mode: it surfaces as a generic `500` error with `"Encryption failed: ENCRYPTION_KEY environment variable is not set"` the first time any code path touches an encrypted field (e.g. registering a remote R710 device). Set it before first boot, not after something breaks.

4.3 Commonly-set optional variables

Variable	Default	Notes
PORT	8080	The app binds <code>0.0.0.0:\$PORT</code> — see §8.
NODE_ENV	—	Set to <code>production</code> for a real deployment.
NEXTAUTH_URL	unset	Deliberately left commented out in <code>.env.example</code> — leaving it unset allows login from both <code>localhost</code> and any LAN IP the server is reachable on. Only set it if you need to pin a single canonical URL.
FORCE_HTTPS	false	See §7.1.
CORS_ORIGINS	localhost + 192.168.0.0/16 + 10.0.0.0/8	Comma-separated allow-list. Adjust for your actual network if it doesn't fall in those private ranges.
LOG_LEVEL	info	<code>error</code> / <code>warn</code> / <code>info</code> / <code>debug</code> .
SYNC_*	—	Only relevant if this server participates in multi-branch peer sync — see <code>.env.example</code> 's SYNC section and <code>SERVICE_*</code> block. Irrelevant for a single-server deployment; leave as placeholders.

4.4 A separate set of variables used only by the installer

`scripts/install/install-database.js` provisions the database using its **own** set of env vars — it does **not** read `DATABASE_URL`:

```
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
POSTGRES_DB=multi_business_db
POSTGRES_SUPERUSER=postgres
```

If you use the automated installer (§8.1) with a custom database name/user, set these to match whatever you then put in `DATABASE_URL` — the two are not automatically kept in sync.

5. Database setup

You can do this manually (below) or via the automated installer, `npm run install:full` (§8), which wraps the same steps. Manual steps, for full control:

5.1 Create the database and user

```
-- as the postgres superuser
CREATE DATABASE multi_business_db;
CREATE USER app_user WITH ENCRYPTED PASSWORD 'a-real-password';
GRANT ALL PRIVILEGES ON DATABASE multi_business_db TO app_user;
```

Put the resulting connection string in `.env.local` as `DATABASE_URL`.

5.2 Install dependencies and generate the Prisma client

```
npm install
npm run db:generate # equivalent to: npx prisma generate
```

5.3 Apply migrations

Always use `prisma migrate deploy` for a real deployment — never `prisma migrate dev`. `migrate dev` is a development-only command that can create a shadow database and prompt interactively; `migrate deploy` just applies the existing migration history non-interactively, which is what you want here. As of this branch there are **453 migration files** in `prisma/migrations/` — this will take a little while on a completely empty database.

```
npm run db:deploy
```

(`db:deploy` is the `.env.local`-aware wrapper described in §4.1; equivalent to `npx prisma migrate deploy` with `DATABASE_URL` exported manually.)

5.4 Seed reference data — required, not optional

A migrated-but-unseeded database is **not usable** — dropdowns, ID-number templates, phone/date format templates, job titles, compensation/benefit types, business categories, and permission templates all come from seed data, not from the schema itself. Run:

```
npm run setup:production -- --no-admin --ignore-missing-models
# equivalent to: node scripts/production-setup.js --no-admin --ignore-missing-models
```

`--no-admin` is important here — see §5.5 for why.

Do **not** confuse this with the various `npm run seed:*` scripts (`seed:hardware`, `seed:clothing-bales`, `seed:customers`, etc.) — those are **demo/sample data** for testing and training environments, not required for a production deployment. Skip them unless you specifically want sample data.

5.5 Create the first admin user

```
npm run create-admin
# equivalent to: node scripts/create-admin.js
```

This creates:

```
email:    admin@business.local
password: admin123
```

Change this password immediately after your first successful login — nothing in the app forces a password change on first login, and this is a well-known default.

Why `--no-admin` matters in §5.4: `scripts/production-setup.js` can *also* create its own admin user if run without `--no-admin` — a second, independent code path (`createSystemAdmin()`) that uses a different internal ID and a different, larger default permissions object than `scripts/create-admin.js` does. Both are functionally fine on their own (neither creates duplicates — both check for an existing user by email first), but running both against the same database is pointless and potentially confusing. **Stick to the sequence above:** `production-setup.js --no-admin` for reference data, then `create-admin.js` for the admin user, exactly as the automated installer does internally.

6. Build

```
npm run build
```

This runs, in order:

- 1. `prisma generate`
- 2. `next build` (with `NODE_OPTIONS=--max-old-space-size=8192` — the Next.js build genuinely needs a large heap on this codebase; don't strip that flag if you're scripting this yourself)
- 3. `npm run build:server` → `tsc --project tsconfig.server.json`, which compiles `server.ts` (and everything it transitively imports) to `dist/server.js`. **This step must succeed before `npm start` will work at all** — `npm start` runs `dist/server.js` directly, and that file only exists after this step.
- 4. `postbuild` (automatic): `scripts/mark-build-complete.js` (writes build markers, including the current git commit — this is what the Windows Service wrapper later checks for staleness) and then `scripts/build-agent.js` (builds the separate R710 local-agent `.exe/.zip` used for remote device support — **failure here is logged but does not fail the overall build**; if you don't need the R710 remote-agent feature you can ignore a warning from this step).

6.1 Verifying the build

```
ls dist/server.js      # must exist
ls .next/BUILD_ID      # must exist
```

If either is missing, `npm start` (§8) and the Windows Service wrapper (§8.2) will both fail.

7. Server-side certificates

There are **two entirely separate certificates** involved in a full deployment, generated and installed differently, for different purposes. Don't conflate them:

	TLS certificate (§7.1)	QZ Tray signing certificate (§7.2)
Purpose	Encrypts traffic between browsers and the app server (HTTPS)	Lets the app's print requests be trusted by QZ Tray on a workstation without a popup every time
Where it's generated	Anywhere with <code>mkcert</code> (often your own workstation, then copied to the server)	On the app server itself , via a script in this repo
Where it's installed	<code>certs/</code> folder at the app's repo root	<code>.env.local</code> (the key pair) and QZ Tray's own trust store on every workstation that prints via QZ Tray
Who needs to trust it	Every client browser (once each)	QZ Tray itself, on every printing workstation (once each)

7.1 TLS / HTTPS (browser ↔ server)

`server.ts` auto-detects HTTPS — there's no on/off flag. On startup it scans a `certs/` folder (git-ignored; does not exist on a fresh clone) for a `*.pem` file (excluding anything ending in `-key.pem` or named `rootCA.pem`) with a matching `<name>-key.pem`. If both are found, it starts HTTPS on the same port; otherwise it starts plain HTTP and logs that it did so. `NODE_ENV` has no effect on this decision.

Why you probably want this anyway

Chrome and Edge enforce **Private Network Access** restrictions that block a plain-HTTP page from talking to QZ Tray (which listens locally for print jobs) unless the page itself is served over HTTPS with a certificate the browser trusts. Firefox has no such restriction and needs none of this. If you

don't use QZ Tray printing (e.g. every printer goes through the Windows Raw Printer path or an agent-relayed printer), you can skip this subsection entirely and run HTTP-only.

Installing mkcert itself

`mkcert` is not preinstalled anywhere and isn't a repo dependency — install it once on whichever machine will generate the certificate (usually the server itself, or any machine you trust to hold the private key briefly before copying it over).

Windows (PowerShell) — no package manager required, it's a single portable `.exe`:

```
New-Item -ItemType Directory -Force -Path "C:\mkcert" | Out-Null
Invoke-WebRequest -Uri "https://github.com/FiloSottile/mkcert/releases/download/v1.4.4/mkcert-v1.4.4-windows-amd64.exe" -OutFile "C:\mkcert\mkcert.exe"

$env:Path += ";C:\mkcert"
[Environment]::SetEnvironmentVariable("Path", $env:Path, "User")
```

Close and reopen PowerShell, then confirm with `mkcert -version`. (Check <https://github.com/FiloSottile/mkcert/releases/latest> first in case a newer version exists — the URL above pins v1.4.4.) If you have Chocolatey or Scoop already, `choco install mkcert` / `scoop install mkcert` works too and skips the manual PATH step.

Linux (Debian/Ubuntu):

```
sudo apt-get update && sudo apt-get install -y libnss3-tools wget
wget -O mkcert https://github.com/FiloSottile/mkcert/releases/download/v1.4.4/mkcert-v1.4.4-linux-amd64
chmod +x mkcert
sudo mv mkcert /usr/local/bin/mkcert
mkcert -version
```

`libnss3-tools` is what lets `mkcert -install` also register the CA with Chrome/Chromium's own NSS trust store, not just the system one — skip it and Chrome-based browsers on that machine won't trust certificates `mkcert` issues. (Other distros: swap `apt-get` for your package manager — `nss-tools` on Fedora/RHEL, or use your distro's `mkcert` package directly if it has one, e.g. `sudo dnf install mkcert` / `sudo pacman -S mkcert`.)

Generating and installing the TLS certificate

Prefer the scripted path (`npm run cert:generate` / `npm run cert:install`, documented in [ADMIN-INSTALLATION-GUIDE.md §6](#)) over the manual commands below — it maintains the server IP list in `scripts/lan-server-ips.json`, always copies the correct current `rootCA.pem` from `mkcert -CAROOT` (avoiding the second ⚠ below entirely), and warns if a server's own IP isn't actually covered. The manual steps here remain as the underlying mechanism and for troubleshooting when something's gone wrong.

```
# on the machine running the server, or anywhere with mkcert installed
mkcert -install # only needed once per machine, sets up mkcert's local root CA
mkcert <server-ip-1> <server-ip-2> localhost 127.0.0.1
```

This produces a cert + key pair (e.g. `192.168.0.108+3.pem` / `192.168.0.108+3-key.pem`) **in your current directory only** — see the ⚠ callout below before assuming `rootCA.pem` is also freshly there. Then:

1. Create a `certs/` folder at the repo root (same level as `package.json`) and place the cert, key, and `rootCA.pem` inside it.
2. Restart the server: `npm run start` (or restart the Windows Service if that's how it's running — see §8.2).
3. Confirm the console (or that service's logs) shows `HTTPS enabled — certs loaded from ./certs/`.
4. The app is now reachable at `https://<server-ip>:8080` (or whatever port you configured).

Trusting this certificate on client machines is covered in §10.1 (workstation setup) — every machine that opens the app in Chrome/Edge needs to do this once.

⚠ List every IP/hostname the server will actually be reached by, in that one `mkcert` command — this is a real, previously-hit failure mode. A cert generated for `192.168.0.108` will silently fail TLS hostname verification if the server is later reached at, say, `192.168.8.166` instead (a different NIC, a DHCP lease change, a new deployment target) — every browser tab and every local-agent socket connection to it fails with a generic, unhelpful error (browsers show a cert warning; a Node-based client like the R710/Workstation local agent just loops on a bare `websocket error` with no further detail). `rootCA.pem` being present and otherwise correct does not save you here — hostname/IP

matching against the *leaf* cert's SAN list is checked independently of CA trust. If the server's reachable address ever changes, regenerate the leaf cert (`mkcert <new-ip> <old-ip-if-still-valid> localhost 127.0.0.1` **on the same machine that already holds the matching CA** — see the next warning if you're not sure that's true), replace the old cert+key pair in `certs/`, restart the server, and **re-pair** any local agents connected to it (revoking and re-pairing is what actually pushes a corrected `caCert` down to an agent — restarting the agent alone does not, since the bad/missing value is already saved in its own config).

⚠️ `mkcert <names>` **never copies `rootCA.pem` into your current directory — only the leaf cert + key.** This is a second, independent failure mode from the one above, and it's easy to trigger by accident: if you run `mkcert <names>` on a machine that has **never run `mkcert -install` before**, `mkcert` silently generates a **brand-new CA** on that machine (stored under its own per-user `mkcert -CAROOT`, e.g. `%LOCALAPPDATA%\mkcert` on Windows) and signs the new leaf cert with it — but nothing copies that new CA's `rootCA.pem` into `certs/`, so whatever `rootCA.pem` was already sitting there (from a previous setup, possibly generated on a completely different machine) is left in place, now signed by a CA that has nothing to do with the leaf cert actually being served. The server still starts and serves HTTPS fine — a browser that's already manually clicked through the warning, or that trusts the OLD CA from a previous setup, won't necessarily notice — but every local agent pairing will fail: the mint endpoint hands the agent the (wrong, stale) `rootCA.pem`, the agent's strict TLS validation rejects the leaf cert it's actually shown, and the agent loops forever on a bare `websocket error` with no further detail, even after multiple re-pairs. To confirm and fix:

```
# confirm the mismatch — compare what the two files actually say
& "C:\Program Files\Git\usr\bin\openssl.exe" x509 -in certs\rootCA.pem -noout -subject -issuer
# if this org/CN doesn't match what the server is actually presenting on the wire, find the CA that
# actually signed the leaf cert:
mkcert -CAROOT
# then copy the correct one over:
Copy-Item "$(mkcert -CAROOT)\rootCA.pem" -Destination "certs\rootCA.pem" -Force
```

Restart the server, then **revoke and re-pair** every local agent connected to it — same reasoning as above, a re-pair is what actually delivers the corrected `caCert`. See §10.3's troubleshooting note for the equivalent workstation-side diagnostic steps (how to confirm this is actually the cause before assuming it).

7.2 QZ Tray signing certificate (generated on the server)

This is the certificate the user-facing docs refer to as "installing our own certificate" — it's separate from the TLS certificate above, generated **on the server** by a script already in this repo, and only matters if you're printing through QZ Tray (§10.2).

By default, QZ Tray shows the operator a trust popup on every single print job unless the request is cryptographically signed with a certificate QZ Tray has been told to trust. Generating your own certificate (rather than using QZ's bundled demo certificate) means that trust is set up once, deliberately, rather than relying on a certificate anyone else with the QZ demo files could also produce.

On the server, one-time:

```
npm run qz:generate-cert
# equivalent to: node scripts/generate-qz-cert.js
```

This requires `openssl` on PATH (see §2) and:

1. Generates a 2048-bit RSA key pair and a self-signed certificate (10-year validity) with `openssl`.
2. Writes the public certificate to `certs/qz-certificate.pem`.
3. Prints two lines to the console — `QZ_PRIVATE_KEY=...` and `QZ_CERTIFICATE=...` (both base64-encoded) — **copy both into `.env.local`**. These are what the server uses to sign every print request it sends to a workstation's QZ Tray.
4. Restart the server (or the Windows Service — `npm run service:restart`) so it picks up the new env vars.

At this point the server is signing its requests, but no workstation trusts the certificate yet — that half of the setup is per-workstation and covered in §10.2.

8. Running the server

There are three ways to run this in production, in increasing order of robustness. Pick one.

Option A — Plain foreground process (quick test only)

```
npm run build    # if not already built
npm run start
```

This runs in the foreground and stops when the terminal closes. Fine for a smoke test; not a real production setup (no restart-on-crash, no auto-rebuild).

Option B — The automated installer (`npm run install:full`)

`node scripts/install/install.js` runs a **non-interactive**, six-step pipeline:

- 1. `preInstallationChecks()`
- 2. `installDependencies()` (`npm install`)
- 3. `installDatabase()` — everything in §5, automated (skip with `--skip-database`)
- 4. `installService()` — installs the **sync** service (see the naming warning in §8.1 below); skip with `--skip-service`
- 5. `postInstallationSetup()`
- 6. `finalVerification()`

```
npm run install:full
# or, to skip pieces you're handling manually:
npm run install:full -- --skip-database --skip-service
```

What this does NOT do: it does not run `npm run build` (§6), and it does not touch `certs/` (§7). You still need to run the build yourself and set up certificates if you want them, whether or not you use this installer.

8.1 ⚠ Naming trap: two different things are both called "the service"

This repo has **two independently-installable Windows services, and their names are actively misleading:**

What you run	Windows service name	What it actually does
<code>scripts/install/install-service.js</code> (installed as part of <code>install:full</code> 's step 4, or standalone)	<code>multi-business-sync</code>	The peer-to-peer database sync background service. Optional. Irrelevant if you're running a single, non-syncing server.
<code>npm run service:install → node windows-service/force-install-hybrid.js</code>	<code>MultiBusinessSyncService</code> (display name "Multi-Business Sync Service")	Despite the name, this is the main application server launcher — see §8.2. It does not primarily do sync; the name is a legacy artifact.

Don't assume "I installed the service" from one of these covers the other. For a normal single-server production deployment, you want **§8.2's** service, not §8.1's.

8.2 Option C — The hybrid self-healing Windows Service (recommended for production)

```
npm run build:service    # prepares windows-service/ assets
npm run service:install  # must run from an elevated/Administrator shell
```

This registers a Windows service (internally named `MultiBusinessSyncService` — see the warning above) whose target script is `windows-service/service-wrapper-hybrid.js`. On every start, that wrapper:

- 1. Checks whether `dist/server.js` and `.next/BUILD_ID` exist, and compares the git commit recorded at last build time (written by `mark-build-complete.js`, §6) against the currently checked-out commit.
- 2. If the build is missing or stale relative to the current commit, **it runs `npm run build` itself** and streams the output, before proceeding.
- 3. Spawns `node -r tsconfig-paths/register dist/server.js` as a monitored child process, restarting it automatically on crash.

Practical implication: with this service installed, a normal deployment update is just:

```
git pull
npm run service:restart
# no need to manually rebuild first – the wrapper detects the new commit
# and rebuilds automatically on that start.
```



```
# (equivalent manual form: open the Services console, or
# net stop MultiBusinessSyncService && net start MultiBusinessSyncService)
```

Prerequisites for this step:

- Run from an elevated (Administrator) shell — `node-windows` needs elevation to register a service with the Service Control Manager.
- PostgreSQL must be installed as the literal Windows service `postgresql-x64-18` (§2), or edit `windows-service/config.js`'s `dependencies` array to match your actual PostgreSQL service name before installing — otherwise Windows will refuse to start this service because its declared dependency doesn't exist.

Linux

There's no equivalent of §8.2's auto-rebuilding wrapper. Run the plain systemd unit `scripts/install/install-service.js` generates `(installUnixService())`, or write your own — either way, it starts `node dist/server.js` directly with no auto-build-on-stale-commit behavior, so **you are responsible for running `npm run build` yourself after every `git pull`**. Stop the service *before* building, not after — `npm run build` (and `npm install`, via its `postinstall` hook) both run `prisma generate`, which needs to overwrite the Prisma query engine binary; a still-running service process holding that file open makes the build fail outright, not just warn.

9. Networking

- The app binds `0.0.0.0:$PORT` (`server.ts`), default port **8080**, directly — not `127.0.0.1`, so it's reachable from other machines on the network as soon as it's running and the firewall allows it.
- **No reverse proxy config is checked into this repo** — no nginx, IIS, Caddy, or similar. If you want one in front of the app (for a stable hostname, additional TLS termination, rate limiting, etc.), that setup is entirely up to you; the app has no expectations about being behind one.
- `CORS_ORIGINS` (§4.3) needs to include whatever origin(s) the app is actually accessed from — the shipped default only covers `localhost`, `127.0.0.1`, and the `192.168.0.0/16` / `10.0.0.0/8` private ranges.
- Tailscale, if you see it mentioned anywhere in ops discussion for this project, is an operational choice for a specific site's remote access — it is not referenced anywhere in the application code or install scripts and is not a deployment requirement.

9.1 Firewall ports for the peer-to-peer sync service

Each server also runs a peer discovery/sync service alongside the main app (§8.2/8.3), which needs its own inbound firewall rules — separate from the app's own port 8080:

Port	Protocol	Purpose
<code>SYNC_PORT</code> (default 8765)	TCP	Sync data transfer between peer servers
5353	UDP	mDNS peer discovery (multicast)

Automated (PowerShell, as Administrator):

```
New-NetFirewallRule -DisplayName "Multi-Business Sync Service" -Direction Inbound -Protocol TCP -LocalPort 8765 -Action Allow
New-NetFirewallRule -DisplayName "Multi-Business Sync Discovery" -Direction Inbound -Protocol UDP -LocalPort 5353 -Action Allow
```

Manual (Windows Defender Firewall with Advanced Security):

1. **Inbound Rules** → **New Rule** → **Rule Type: Port**
2. **Protocol:** TCP, **Specific local ports:** `8765` → Allow → all profiles → name it "Multi-Business Sync Service"
3. Repeat for **UDP** port `5353`, named "Multi-Business Sync Discovery"

Verify from another machine on the LAN:

```
Test-NetConnection -ComputerName <this-server-ip> -Port 8765
```

9.2 Initial Load — syncing an existing server's data to a brand-new one

If a new server should start with a copy of an **existing** server's data (rather than a fresh empty database, or a manually restored `pg_dump`), the sync service has a built-in one-way transfer for exactly this — separate from its normal ongoing peer-to-peer sync, and **never triggered automatically**

by deployment, a `git pull`, or a service restart.

Deploy the new (target) server first, as a completely normal fresh install (§2–§8) — empty database, migrated and seeded, service running. Do not run Initial Load until the target is actually up and reachable.

Then, from the Admin UI on the SOURCE server (the one that already has the data):

1. Open `http://localhost:8080/admin/sync`
2. Click the **Initial Load** tab
3. Select the target peer from the list
4. Click **Start Initial Load** and monitor progress in real time

Advanced/scripted equivalent — `POST` to the source server's own API:

```
curl -X POST http://localhost:8080/api/admin/sync/initial-load \
-H "Content-Type: application/json" \
-d '{"action": "initiate", "targetPeer": {"nodeId": "<target-node-id>", "ipAddress": "<target-ip>"}}'
```

Common mistake: triggering this as part of a deployment script (`git pull && npm run service:restart && curl ../initial-load`) — always deploy, verify the target is actually up, *then* trigger the load as a separate manual step from the source.

10. Workstation setup

Everything above is done once, on the server. This section is what needs to happen on each individual machine that will actually use the app — a POS terminal, a back-office PC, or a workstation with a printer/scale attached. Not every workstation needs every piece below — pick the ones relevant to how that particular machine is used.

10.1 Just browsing the app (every workstation)

1. Open the server's URL in a modern browser (Chrome, Edge, or Firefox) — `https://<server-ip>:8080` if TLS is set up (§7.1), otherwise `http://<server-ip>:8080`.
2. **If the server runs HTTPS with a self-signed (mkcert) certificate**, this browser needs to trust it once, or it'll show a security warning (and QZ Tray printing won't work at all in Chrome/Edge until this is done). Copy `rootCA.pem` from the server's `certs/` folder to the workstation (USB, shared folder, email — it's a public file, not a secret) first, then:
 - **Windows, automated:** alongside `rootCA.pem`, copy a small trust-installer script and run it — see the exact one-click pattern already used in `certs/README.md` (`setup-ssl.bat`) if you want to reuse that approach.
 - **Windows, manual:** `Win+R` → `certmgr.msc` → **Trusted Root Certification Authorities** → right-click **Certificates** → **All Tasks** → **Import** → select `rootCA.pem` → restart the browser.
 - **Linux (Debian/Ubuntu):**

```
sudo cp rootCA.pem /usr/local/share/ca-certificates/mkcert-rootCA.crt
sudo update-ca-certificates
```

This covers the system trust store (curl, wget, most non-Chromium apps). For Chrome/Chromium specifically, which uses its own NSS trust store, either install `libnss3-tools` and run `mkcert -install` locally using the *same* `rootCA-key.pem` (if you have it — not just the public `rootCA.pem`), or import it manually via `chrome://settings/certificates` → **Authorities** → **Import**. Firefox on Linux also uses its own store — `about:preferences#privacy` → **View Certificates** → **Authorities** → **Import**.

- **Firefox (any OS)** needs none of this for the *page itself* to load once you click through the one-time warning — it doesn't enforce the Private Network Access restriction that makes browser-level trust a hard requirement for QZ Tray in Chrome/Edge. Still recommended for a clean, warning-free experience either way.
3. Log in with a user account (the first admin account is created per §5.5 — change its password immediately if you haven't already).

10.2 Printing via QZ Tray (workstations with a locally-attached printer, using the QZ Tray path)

1. Install **Java 8+** if not already present (`https://java.com`) — QZ Tray requires it.
2. Install **QZ Tray** from `https://qz.io/download/` and leave it running (it starts with Windows by default, and sits in the system tray).
3. Make sure the printer itself is installed as a normal Windows printer first (see §10.4 if it isn't yet).
4. **Trust the server's signing certificate** (the one generated in §7.2, not the TLS one) — this is the "install our own certificate" step:
 - Copy `certs/qz-certificate.pem` from the server to this workstation (USB, shared folder, email — it's a public certificate, not a secret).
 - Right-click the QZ Tray icon in the system tray → **Advanced** → **Site Manager** → **Add Certificate** → select `qz-certificate.pem`.

- From then on, print requests signed by the server with the matching `QZ_PRIVATE_KEY/QZ_CERTIFICATE` (§7.2) are trusted silently — no per-job popup.
5. In the app's POS receipt preview modal, the printer dropdown should now list this workstation's locally-installed printers (detected by QZ Tray). If it instead says "QZ Tray not detected," confirm QZ Tray is actually running in the system tray.

If you skip generating your own certificate (§7.2) and just want something working quickly on a trusted LAN, QZ Tray ships a demo certificate (`demo-signing.crt/demo-signing.key`, inside its own install folder) that can be used the same way — but because it's the same demo cert everyone gets, it's not something to rely on beyond a quick test.

10.3 R710 remote WiFi devices, or a remote scale/printer (workstations that need the local agent)

Only needed for: (a) an R710 WiFi controller that isn't on the same local network as the app server, or (b) a scale/printer physically attached to a workstation the server can't reach directly. If neither applies to a given workstation, skip this.

1. From the relevant device's **Agent panel** in the app (R710 Portal → Devices → Agent, or Admin → Workstation Agents), download `r710-agent.zip` — the download link is always available there, not just during initial setup.
2. Unzip it on the workstation and run `r710-agent.exe`. No installer, no separate Node.js/runtime needed — it's a self-contained executable. A tray icon appears.
3. From the browser, **on that same workstation**, open the Agent panel again and click **Pair this machine**.
4. Right-click the tray icon → **Preferences** → **Start with Windows** so it survives a reboot without manual intervention.
5. One agent install can be paired to R710, a scale, a printer relay, or any combination, and even to more than one server at once — see `docs/user-guide.md`'s R710/Workstation Agent sections for the full day-to-day usage, pairing screenshots, and troubleshooting detail; this guide only covers that it needs installing.

Troubleshooting: agent pairs successfully but immediately loops on a bare `websocket` error

If the agent's console shows the connection state cycling `connecting` → `error` → repeat, forever, and this doesn't clear up even after revoking and re-pairing, suspect a TLS certificate mismatch on the server side (§7.1's second ⚠️ callout) rather than anything wrong on the workstation. The agent (unlike a browser) cannot be manually told to trust an untrusted certificate — a mismatch there fails silently and permanently, with no more specific error surfaced.

To confirm it's a cert issue and not something else (firewall, wrong port, DNS), from the **workstation**, in PowerShell, test the raw TLS handshake directly against the server, independent of the agent entirely:

```
" " | & "C:\Program Files\Git\usr\bin\openssl.exe" s_client -connect <server-ip>:8080 -servername <server-ip>
```

Look at the `issuer=` line in the output — that names the machine whose mkcert CA actually signed what's being served. Then compare that against the actual current `certs\rootCA.pem` **on the server** (not a copy sitting on this or any other workstation — a stale local copy of that file is a common way to mislead this exact check):

```
& "C:\Program Files\Git\usr\bin\openssl.exe" x509 -in "certs\rootCA.pem" -noout -subject -issuer
```

If the `issuer=` from the live TLS handshake doesn't match the `subject=/issuer=` of the server's own `certs\rootCA.pem`, that confirms the exact scenario in §7.1's `mkcert` gotcha — fix it there (on the server), then come back and re-pair this agent. Restarting the agent alone will never fix this; the wrong `caCert` is already saved in its own pairing file, and only a fresh pairing (Revoke Pairing → Pair this machine again, no re-download needed) delivers a corrected one.

10.4 Receipt printer drivers (workstations printing via a "regular Windows printer," not QZ Tray/agent-relayed ESC/POS)

If a thermal printer needs to show up as a normal installed Windows printer (rather than being addressed directly over ESC/POS via QZ Tray or the local agent), it needs a Windows driver first. This repo has device-specific walkthroughs already:

- `INSTALL_EPSON_TM-T20III_DRIVER.md` (repo root) — EPSON TM-T20III
- `INSTALL_RONGTA_DRIVER.md` (repo root) — RONGTA 80mm series

Both boil down to: install the manufacturer's Windows driver (or let Windows auto-detect it under **Settings** → **Devices & Printers** → **Add a printer**), confirm it appears in **Devices and Printers**, then select it from the app's printer configuration (Admin → Network Printers, or the POS receipt preview modal, depending on connection mode).

10.5 Electron dual-monitor POS kiosk (optional, customer-facing display setups only)

If a POS terminal needs a second, customer-facing monitor showing a live order/cart display in kiosk mode (no browser chrome, can't be accidentally closed), that workstation can run the bundled Electron shell instead of a plain browser tab.

This no longer has to run on the same machine as the server. Earlier versions hardcoded `localhost` as the only server address Electron would ever load — Electron now registers one or more servers by address (any workstation on the network, or several different servers entirely — see §11.7's admin flow, the same idea applied here) and connects to whichever one is selected, with each server's login kept completely isolated from the others via its own session partition. Certificate trust for a self-signed server cert is handled inside Electron itself at registration time (trust-on-first-use, then pinned to that exact certificate) — separate from, and not requiring, the browser-side `rootCA.pem` trust dance in §10.1.

Quick path — running from source (needs Node and a full checkout of this repo on the workstation, same as any dev environment):

```
cd electron
npm install
npm start
```

On first launch (or if no server has been registered yet), a local "Select Server" screen appears — nothing remote loads until a server is added and its connection is verified.

Production path — a standalone installer, no Node or repo needed on the workstation (recommended for an actual kiosk, same reasoning as shipping `r710-agent.exe` as a standalone `.exe` rather than asking a workstation to run from source):

```
cd electron
npm install
npm run build:win      # or build:mac / build:linux
```

This produces an installer in `multi-business-electron-dist/` — a sibling folder next to the repo root, not inside it (see `electron/scripts/stop-running-app.js` for why: keeping the build output outside the repo avoids a Windows file-lock conflict with VS Code's file watcher). Copy just that installer to the target workstation and run it — no checkout of this repo, no Node.js, no npm install needed there at all. Before building for real distribution:

- **Supply real icon files** — `electron/icon.ico` (Windows), `icon.icns` (macOS), `icon.png` (Linux) are referenced in `electron/package.json`'s build config but don't ship with this repo; `electron-builder` will fall back to its own placeholder icon without them, which is fine for testing but not for something handed to a business.
- **The installer is unsigned** unless you separately obtain and configure a code-signing certificate — Windows SmartScreen will show an "unrecognized app" warning on first run (the same "More info → Run anyway" click-through most unsigned internal tools require). Not a bug, just worth setting expectations for whoever's installing it.

The Windows installer is configured as a **single, per-machine install** — installed once, available to every Windows user account on that workstation (Program Files, shortcuts in the all-users Desktop/Start Menu), not just whoever happened to run the installer. This means:

- It's a **one-click install** — no wizard, no install-location picker, nothing to configure. Run it and it installs, creates both shortcuts, and launches itself.
- Because it writes to Program Files and the all-users shortcut locations, Windows will show a **UAC elevation prompt** ("Do you want to allow this app to make changes to your device?") during install — expected for a per-machine install, not an error.
- **Running the installer over an existing install force-closes any running instance of the app first** (`electron/installer.nsh`, an NSIS `customInit` hook) — upgrading a workstation while the app happens to be open won't fail with a "file in use" error the way it otherwise would; it just closes and gets replaced.

Either way, once running, registering the actual server the workstation should use happens entirely inside the app's own "Select Server" screen — no environment variables, no config file to hand-edit.

10.5.1 Registering and switching servers

This kiosk can hold several registered servers at once — different companies, or a test server alongside a real production one — and switch between them, each with a **completely separate login**. Server B never sees Server A's cookies or session, even mid-shift on the same kiosk.

Adding a server:

1. Click **+ Add Server** (the first time, you'll be asked to set a local PIN — this protects add/remove only; switching between already-registered servers never needs it).
2. Enter a label, the server's IP address, and an **admin** email/password for that server.
3. Click **Test Connection** — a real connection attempt, not a ping: validates the IP, checks reachability, and actually signs in with the given credentials to confirm they're a real admin account on that specific server. If the server presents a self-signed certificate that isn't already

- trusted, its details are shown and trust must be explicitly confirmed before the test proceeds — that exact certificate is then pinned to this one server going forward, never blanket-trusted by hostname.
- The credentials are used only for this one-time check and are never stored — whoever actually uses the kiosk logs in fresh, with their own account, once the server is open.
 - Save Server** only enables once a test has passed for the exact values currently in the form — editing anything afterward requires re-testing.
 - A **duplicate IP/host is rejected** — if another registered server already uses the exact address you entered, Save fails with the name of the existing entry instead of creating a second registration for the same machine. Use **Edit** on the existing one instead.

A bare IP expands to `https://<ip>:8080` (this app's standard port) by default; use **Advanced: use a full URL instead** for a custom port or a different setup. If the server being registered runs on this same machine, check **This server runs on this machine (localhost)** instead of typing it — the IP field fills itself in and locks. Loopback addresses (`localhost`, `127.0.0.1`) are genuinely ambiguous — a bare `npm run dev` instance is plain HTTP, but a full production-configured install running locally is HTTPS-only on that same port — so the test tries `https://` first and only falls back to `http://` if that specific attempt can't reach the server at all; a real LAN IP always defaults straight to `https://`, as before.

Editing a server: click **Edit** next to any registered server (PIN-gated, same as add/remove) to open the same form pre-filled with its current label, IP/URL, and support contact — admin credentials are never stored, so re-verifying them via **Test Connection** is required before **Save Changes** enables, exactly like adding a new one. Use this when a server's IP address changes, rather than registering a duplicate entry: changing the IP re-derives that server's internal id, which carries its default-business setting and other saved details across to the new address automatically.

Switching servers: click **Connect** next to any registered server, or use the **Server → Switch Server...** menu item (bound to `Ctrl+Shift+S` / `Cmd+Shift+S`, which works even with the menu bar hidden in kiosk mode). Switching briefly tears down and recreates both windows — deliberate, not a bug, since a session partition can't be swapped on a live window without risking leaking state between servers.

The server this kiosk is currently connected to is highlighted blue in the list with a "Connected" tag, and its **Connect** button is disabled — reconnecting to the server you're already on is a no-op, and the highlight doubles as an at-a-glance "you are here". If a connection attempt to some other server just failed, that server's row is highlighted amber with an "Unreachable" tag instead (see §10.5.2).

Every later launch automatically reopens whichever server was used last — the picker only reappears if that server can't be reached (with the failure and its registered support-contact number shown right there), if it's been removed, or if you explicitly switch back to it.

Kiosk shell version: the landing and sign-in screens show a small badge — "Connected to: <server> · v<version>" — naming the installed build of this kiosk shell (`electron/package.json`'s version), so an operator can tell at a glance without opening any menu. It turns amber with an "update available" tag if this install is older than what the connected server's own checkout currently expects (read from that server's own `electron/package.json` — same idea as the r710-agent's own update check in §11).

On every launch (and every server switch), a brief "Connecting to <server>..." spinner screen shows while the app reaches out to that server — this can take a little while against a slow connection, and considerably longer if the server has actually gone unreachable (there's no engineered timeout; it's whatever the OS's own network stack takes to give up). The spinner is there so that wait reads as "still connecting," not "the app has hung" — if the connection does eventually fail, the picker reopens with the failure banner as described above.

10.5.2 Troubleshooting

Problem	Likely cause / fix
Stuck on the server picker / "Couldn't reach <server>"	Confirm the server is actually running and reachable from this machine (try its address in a regular browser first). If it moved or changed IP, use that server's Edit button to update its address in place (see below) rather than registering a duplicate entry. Check the support-contact number on the failure banner, if one was set when this server was registered.
"No secondary display detected"	Connect the second monitor before starting Electron, check display settings in the OS, restart Electron after connecting.
Customer display not fullscreen	Normal — kiosk mode handles it automatically once the page finishes loading; use <code>Esc</code> to exit kiosk mode for testing only.
DevTools not opening	Set <code>NODE_ENV=development</code> before launching, or use <code>npm run dev</code> from <code>electron/</code> .
Cart/order updates never appear on the customer-facing monitor	Confirm the business is on a build that includes the customer-display broadcast fix for its POS page — restaurant/grocery/clothing/hardware always had this; vehicle_service, services, vehicles, construction, and consulting (all sharing Universal POS) only got it in a later fix (cart state, payment-complete, and order-cancellation broadcasts all included).

Alternative to Electron entirely: a plain browser on Chrome/Edge 100+ can position a customer display on a secondary monitor itself via the Window Management API — the "Display" button in the POS, with a one-time permission prompt. No install at all, but without kiosk-mode lockdown or dual-monitor auto-detection on launch.

11. Workstation Agent & Printer Architecture

This section explains how receipt printing actually gets from the server to a physical printer, how printers are shared across workstations and businesses, and how to set up and troubleshoot it as an admin. §10.3 above covers installing the agent; this goes deeper into the design behind it. It reflects the architecture as of MBM-283 (multi-business-per-agent) and the zombie-printer cleanup that followed it.

11.1 Core concepts

Term	Meaning
Server	The central app (this Next.js app + database). Runs on one machine, reachable by every workstation on the network.
Business	A tenant in the multi-business app (e.g. "Happy Eater", "Jiffy Lube"). Each has its own printers, users, POS.
Workstation	A physical PC on the local network — a till, a kitchen printer station, an office desktop.
Workstation Agent (<code>WorkstationAgents</code> table)	A <i>pairing</i> between one workstation and one business on one server. Not "one row per PC" — see §11.3.
Local Agent (<code>r710-agent.exe</code>)	The small background process that runs on a workstation, holds one persistent connection per pairing, and relays print jobs + scale readings.
Network Printer (<code>NetworkPrinters</code> table)	One printer record. Can be DIRECT (server talks to it over IP), AGENT (relayed through a workstation agent), or registered for QZ Tray (a separate, browser-driven path — see §11.1a).

11.1a The three printing paths

This app can print three genuinely different ways. They are not interchangeable and a printer only ever belongs to one of them:

1. **DIRECT mode** — the server itself opens a socket to the printer's IP and prints over the network (ESC/POS raw TCP, typically port 9100). No agent involved. Business-agnostic: a DIRECT printer has no `workstationAgentId` and is not scoped to any business.
2. **AGENT mode** — the server sends the print job over the persistent agent socket to a specific paired workstation, which prints to a Windows-installed printer using the local print spooler. This is what the rest of this section is mostly about. Business-scoped: every AGENT printer belongs to exactly one `WorkstationAgents` pairing, which belongs to exactly one business.
3. **QZ Tray** — a completely separate, browser-driven path (§10.2). The *browser*, not the server, talks to QZ Tray running on whatever machine the browser tab is open on, which then prints locally. No agent, no server-side routing, no business scoping — it is purely "whatever this browser last saved as its QZ printer." It exists mainly as a zero-agent-setup fallback for a machine with nothing else configured.

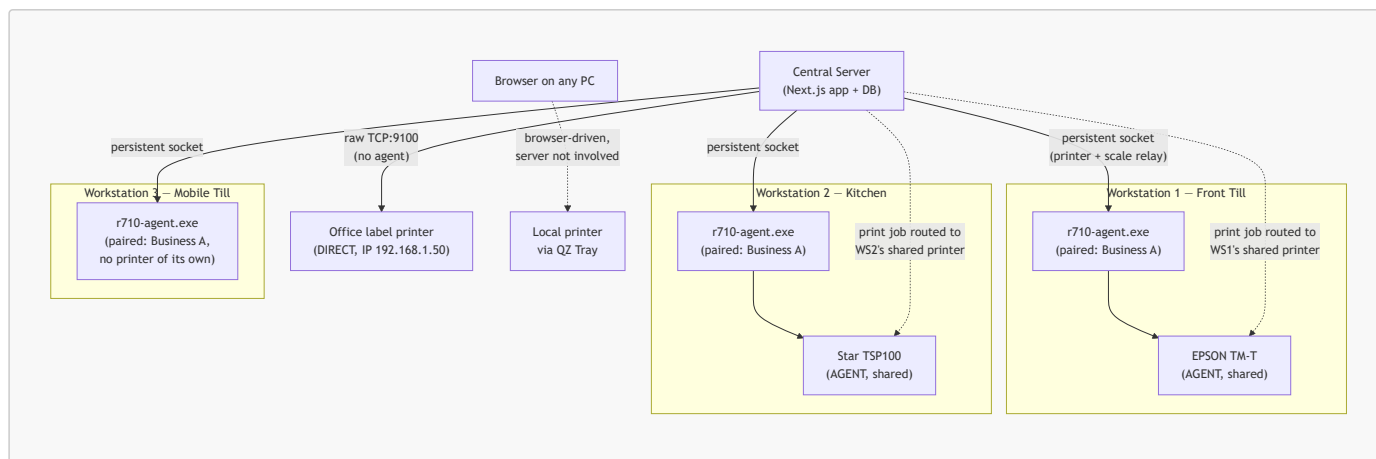
Two printers can share the exact same physical hardware and printer name (e.g. "EPSON TM-T") while being registered under two different modes at once — that is not a bug, just two independent ways of reaching the same box.

11.1b Key `NetworkPrinters` fields

Field	Meaning
<code>connectionMode</code>	'AGENT' or 'DIRECT'.
<code>workstationAgentId</code>	Which <code>WorkstationAgents</code> pairing this printer is physically attached to. Only set for AGENT mode.
<code>remotePrintingEnabled</code>	"Is this printer reachable from the server at all right now?" Off = paused, not deleted — the printer definition is kept, but no job will ever be routed to it.
<code>remoteEnabled</code>	"Share this printer" — can devices <i>other than</i> the workstation it's attached to route to it? Only meaningful (and only ever true) while <code>remotePrintingEnabled</code> is also true.
<code>isShareable</code>	An older, unrelated flag from before AGENT mode existed. Don't confuse it with <code>remoteEnabled</code> — it's a different field with a different history and doesn't gate AGENT-mode routing.

A workstation can **always** print to its own declared AGENT printer even when `remoteEnabled` is off — "share this printer" only controls whether *other* workstations in the business can also route to it.

11.2 Architecture diagram



Every arrow from the Server into an agent is a **persistent, always-on socket connection** — not a one-off request. A print job is one self-contained round trip over whichever socket is relevant; there is no "active workstation" exclusivity for printing (unlike the physical scale, which genuinely can only be owned by one business at a time — see §11.3b).

11.3 Connection lifecycle

11.3a Pairing

Pairing happens once per (workstation, business) combination, from **Admin** → **Workstation Agents**. It mints a token, stores a new **WorkstationAgents** row (**businessId**, **label**, **hostname**, a hash of the token), and the local agent on that machine saves the credential and connects. From then on the agent reconnects automatically on every restart/reboot — no re-pairing needed unless it's explicitly revoked.

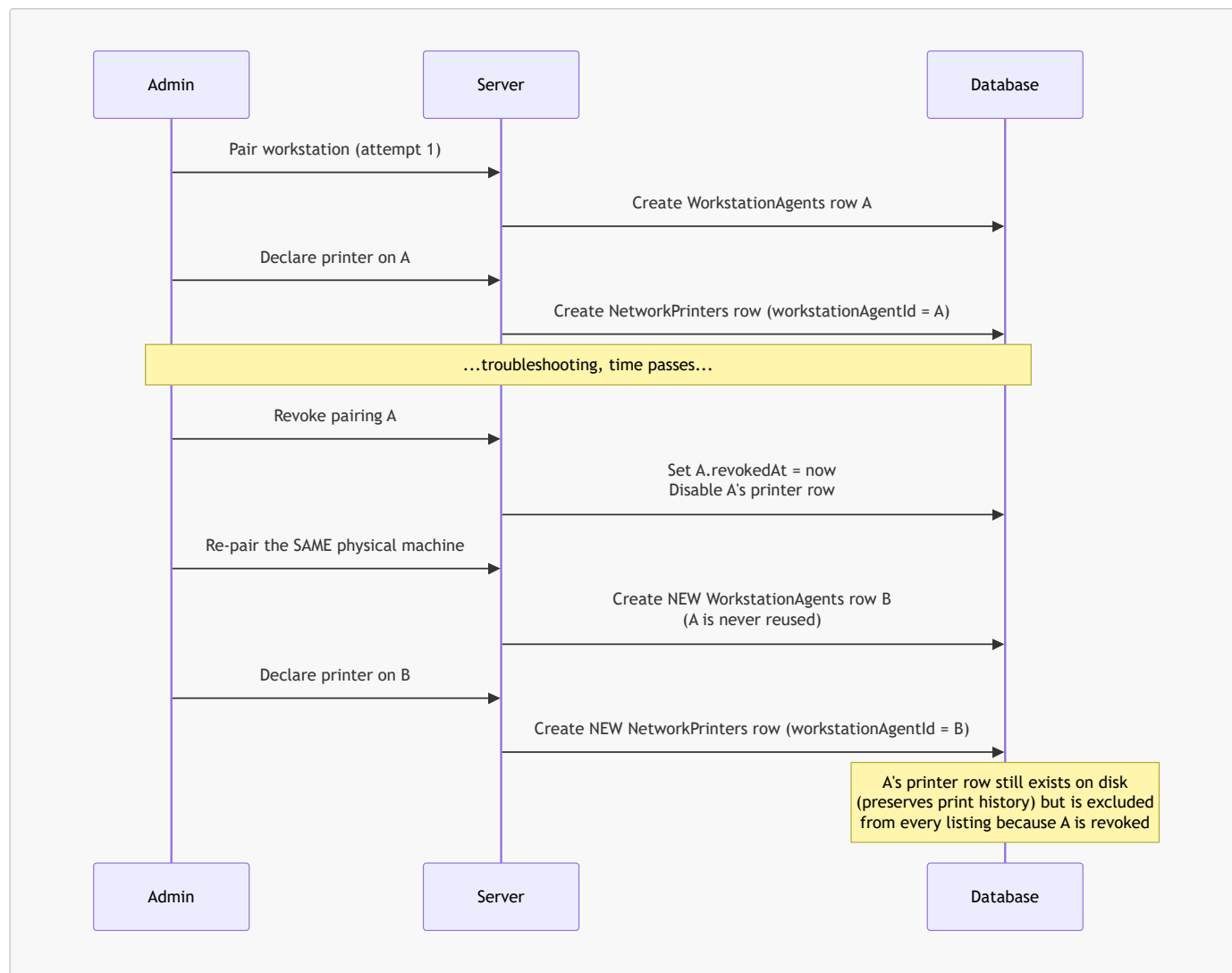
11.3b One physical machine, several businesses

The **same physical PC** can be paired to **more than one business** on the same server — e.g. a shared front desk that rings up both "Happy Eater" and "Jiffy Lube." Each pairing is its own independent **WorkstationAgents** row, its own token, and — critically — its own **persistent socket connection**, all running concurrently in the one **r710-agent.exe** process on that machine.

This matters because of a bug this architecture specifically fixes: an earlier design kept only one such socket "active" per physical machine at a time, and switching which business's browser tab had focus would silently disconnect the other business's printer relay. Today, **every paired business's connection is independent and stays up regardless of which business currently has UI focus** — printing has no exclusivity concern at all, since a print job is a single self-contained round trip and the Windows spooler already queues concurrent jobs safely.

The **only** thing that is genuinely exclusive per physical machine is the attached **scale** (one serial port = one hardware resource). Ownership of the scale is arbitrated separately (whichever business currently has browser/tray focus on that machine), and is unrelated to whether either business's printer relay is connected — both stay connected regardless of who currently "owns" the scale.

11.3c Revoking a pairing



Revoking (Admin → Workstation Agents → Revoke) sets `revokedAt` on the `WorkstationAgents` row and disconnects its socket. It is a soft, permanent end to that specific pairing — re-pairing the same physical machine for the same business afterward creates a **brand-new** `WorkstationAgents` row, never reuses the old one.

Revoking also:

- Deletes any `ScaleDeviceConfigs` row that referenced the pairing (a stale one otherwise makes the app keep retrying a dead scale connection forever).
- Disables (`remotePrintingEnabled = false`, `remoteEnabled = false`) any `NetworkPrinters` row declared on that pairing — **disabled, not deleted**, because deleting would cascade-delete real print job history (`print_jobs`, `default_receipt_printer_configs` both reference `NetworkPrinters` with `onDelete: Cascade`).

On top of that, every printer *listing* the app does — the print-time picker, the admin printer list — independently excludes an AGENT printer whose owning pairing is revoked, regardless of those flags. This is defense in depth: a pairing revoked before this cleanup existed still self-heals the moment the code runs, with no manual database work needed.

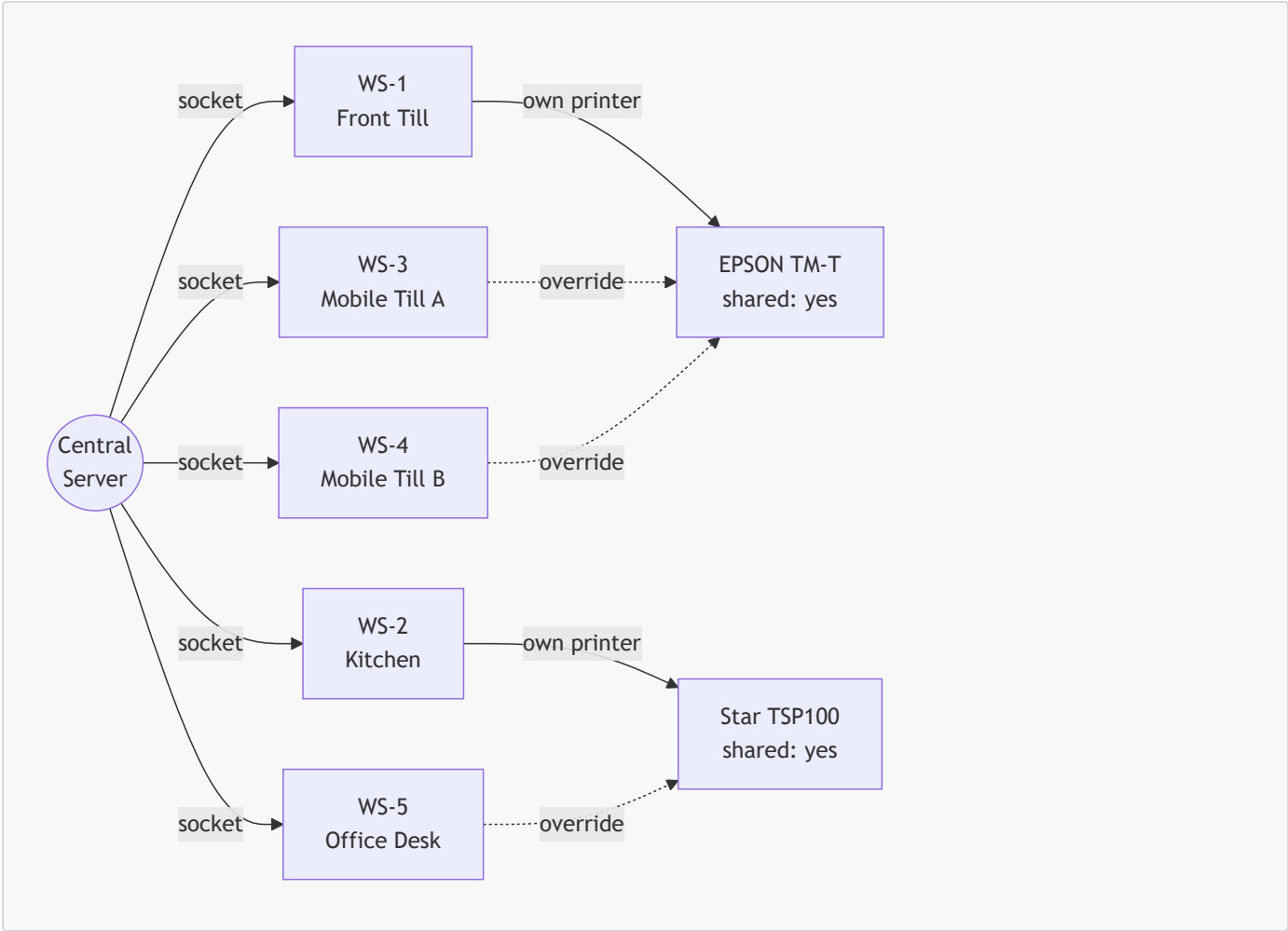
11.4 Business scoping rules

- An **AGENT-mode** printer belongs to exactly one business (via its `WorkstationAgents.businessId`). Only that business's users can print to it, and it only ever appears in that business's printer picker.
- A workstation can **always** discover and print to its own declared AGENT printer, even while `remoteEnabled` (shared) is off.
- A **DIRECT-mode** printer has no business association at all — it's a raw IP the server can reach, available to whichever business's flow happens to be configured to use it. This is true even when the physical machine at that IP is *also* running the server itself — running both roles on one box doesn't change what "DIRECT" means.
- **QZ Tray** printers have no business association either — they're a per-browser, per-machine setting, unrelated to server-side routing entirely.

In short: **business-awareness only exists on the AGENT path**. DIRECT and QZ Tray are both business-agnostic by design, not by omission.

11.5 Worked scenario — one server, five workstations

Setup: one server, five workstations all paired to the **same business**. Two of the workstations each have their own physical printer, set up for remote printing and shared. The other three have no printer of their own — two of them print through one of the shared printers, the remaining one prints through the other.



Solid arrows are a workstation's own declared printer; dashed arrows are a per-workstation default override routing a printer-less workstation to someone else's shared one.

Workstation	Own printer?	remotePrintingEnabled	remoteEnabled (shared)	Who actually prints here
WS-1 (Front Till)	EPSON TM-T	On	On	WS-1 itself, and WS-3 + WS-4 (see below)
WS-2 (Kitchen)	Star TSP100	On	On	WS-2 itself, and WS-5
WS-3 (Mobile Till A)	none	—	—	Routes to WS-1's EPSON TM-T
WS-4 (Mobile Till B)	none	—	—	Routes to WS-1's EPSON TM-T
WS-5 (Office Desk)	none	—	—	Routes to WS-2's Star TSP100

What's actually configured in the database for this:

- **Two WorkstationAgents rows with printers** — WS-1 and WS-2, each with one NetworkPrinters row (`connectionMode: 'AGENT'`, `workstationAgentId` pointing at that row, `remotePrintingEnabled: true`, `remoteEnabled: true`).
- **Three more WorkstationAgents rows with no printer of their own** — WS-3, WS-4, WS-5. They're still paired (so their own socket is live, and they can still receive scale-relay jobs, run local diagnostics, etc.) — they just never declared a physical printer.
- Each of WS-3 and WS-4 has a **per-workstation default printer override** (`DefaultReceiptPrinterConfigs`, keyed by `workstationAgentId`) set to WS-1's printer. WS-5's override is set to WS-2's printer.

What happens at print time on each workstation

The print picker (`UnifiedReceiptPreviewModal`) resolves a default printer in this order (see §11.6 for the full rule and why):

1. This browser's own last-selected printer for *this business* (if still verifiably online).
2. This workstation's admin-set override, if any.
3. This workstation's *own* declared AGENT printer, if any.

4. This business's server-wide default.
5. QZ Tray's saved printer (last resort, unverified).

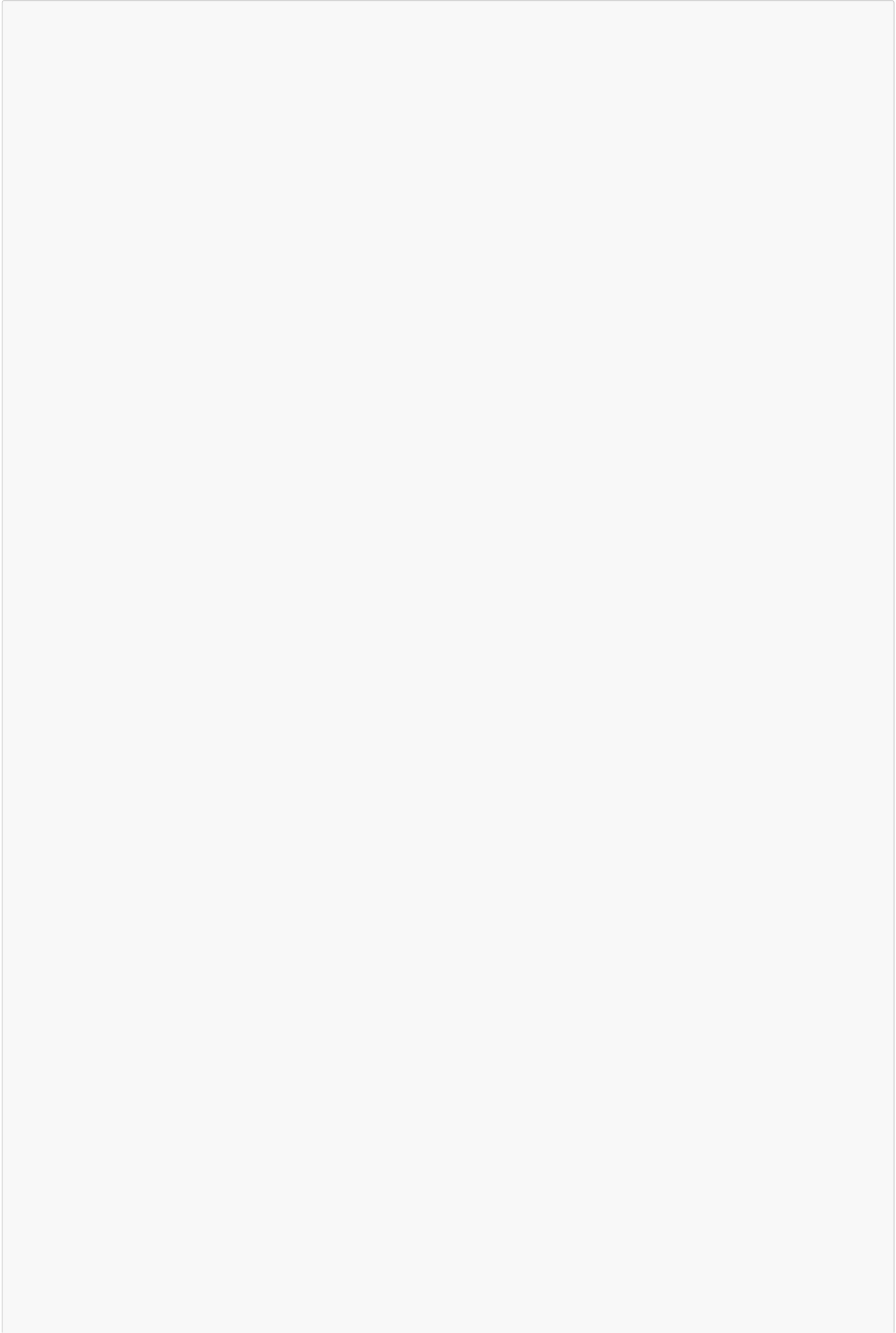
- **WS-1 and WS-2:** step 3 resolves immediately — each prints to its own attached printer without any override needed.
- **WS-3 and WS-4:** no printer of their own, so step 3 finds nothing; step 2's override (set to WS-1's printer) resolves it.
- **WS-5:** same shape — step 2's override (set to WS-2's printer) resolves it.

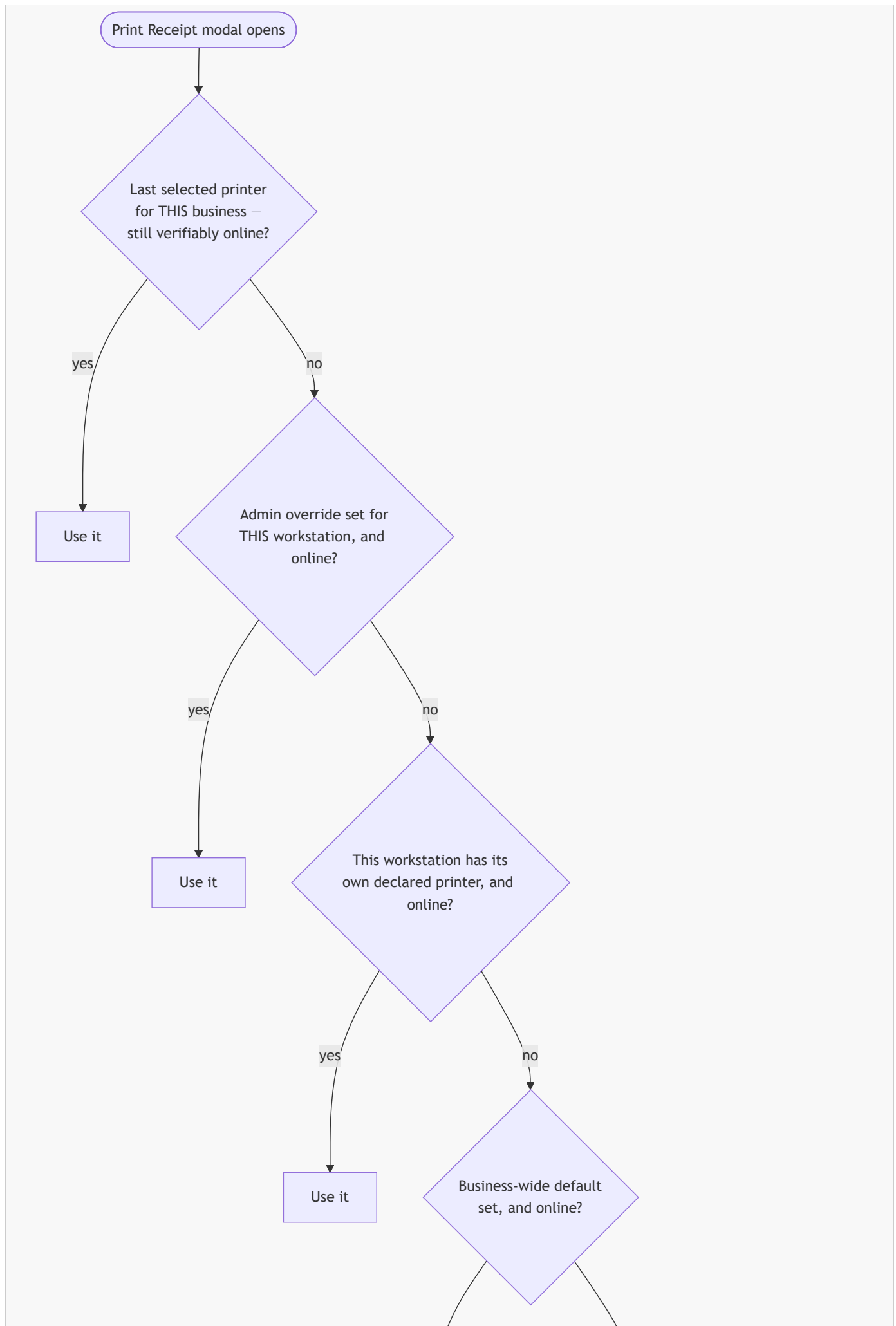
If WS-1's printer goes offline, WS-3/WS-4's override still *points* at it, but the picker's `.isOnline` check skips a dead pick and falls through — worst case, down to the business-wide default (step 4), which an admin should point at whichever shared printer is more reliably up.

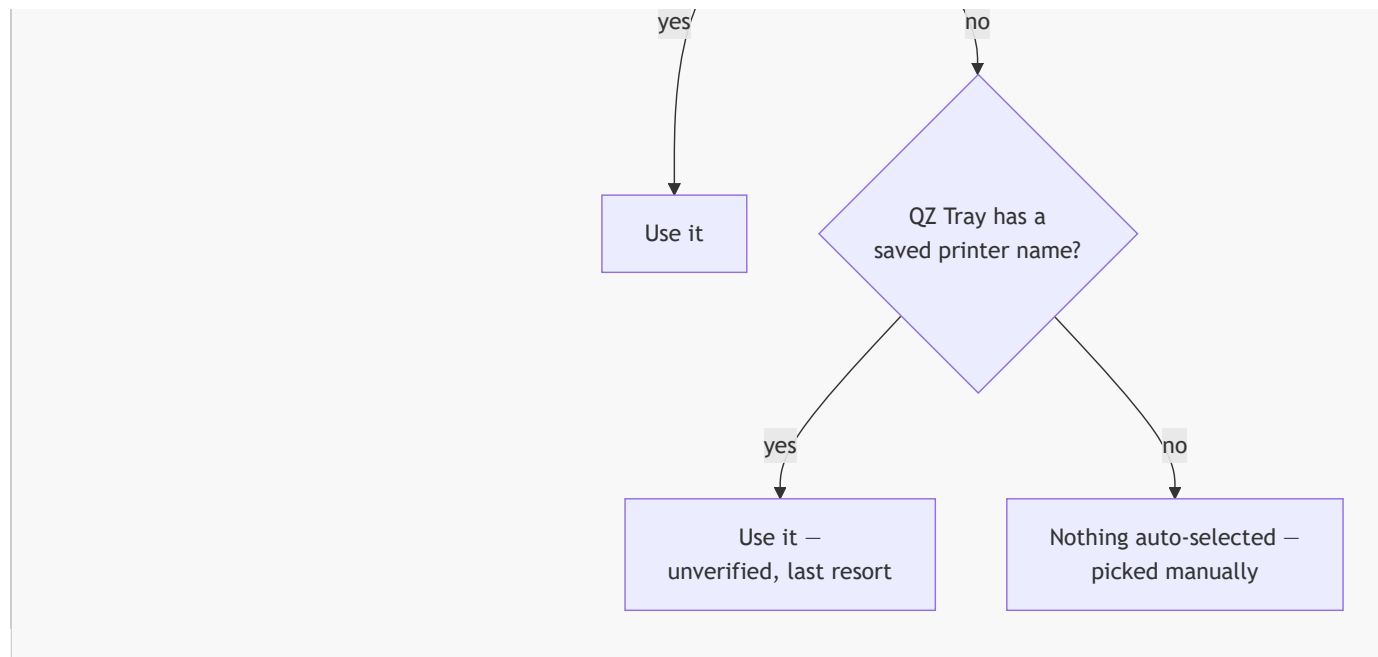
If this were instead spread across two businesses

Nothing above changes conceptually if, say, WS-1/WS-3/WS-4 belong to Business A and WS-2/WS-5 belong to Business B, sharing the same five physical machines and the same server. Each workstation still gets its own independent `WorkstationAgents` row **per business it's paired to** (a machine paired to two businesses has two rows, two tokens, two live sockets — see §11.3b), and AGENT-mode business-scoping (§11.4) means Business A's users never see or can route to Business B's printer, even though both printers are physically reachable from the exact same server.

11.6 Default printer resolution, in detail







Why this order, specifically:

- **Last-selected** wins first because it's the most specific, most recent signal of what a particular person actually wants on a particular machine — but only when it can be *verified* still online. It is scoped per user **and per business** (not globally per user) — a QZ Tray choice made once for one business, or during testing, must never leak into a different business's default.
- **Workstation override** and **own declared printer** both rank above the business-wide default because they're more specific to *this exact machine* — a business default might belong to a completely different workstation.
- **QZ Tray is always last resort**, even when it was the last thing selected, because its "saved printer" is just a remembered *name* — there's no way to verify QZ Tray is actually running without connecting to it (deliberately not done on load, to avoid a permission prompt every time the modal opens). An unverified guess should never outrank something the app can actually confirm is online.

11.7 Admin setup guide

11.7a Pair a new workstation

1. On the target workstation, install and run **r710-agent.exe** (see the download link on **Admin → Workstation Agents**).
2. On that same machine's browser, open **Admin → Workstation Agents** for the business this workstation should serve, and click **Pair**. The local agent's pairing endpoint (<http://127.0.0.1:47710>) is probed automatically — pairing completes with no manual token entry.
3. Repeat step 2, on the same machine, for a **second** business if this workstation should also serve one — this creates a second, independent pairing, not a replacement of the first.

11.7b Declare a printer on a workstation

1. On **Admin → Workstation Agents**, find the paired row for the business + workstation in question, click **Set up printer** (or **Edit** if one already exists).
2. Click **List Printers** to pull every printer Windows currently sees on that workstation (the agent must be online), or type the exact name from Windows' own Printers & Scanners settings.
3. Toggle **Enable remote printing** on — this is what makes the printer reachable from the server at all.
4. Toggle **Share this printer** on if other workstations in this business should also be able to route to it. Leave it off if it should only ever be used by this exact workstation.
5. Click **Save**, then click **Test Print** right there — no need to navigate to a different admin page to confirm it actually works.

11.7c Set a business-wide or per-workstation default

- **Business-wide default:** on the same page, use the "Default printer for this business" picker. This is the fallback used by any device with nothing more specific configured.
- **Per-workstation override:** use "Default printer for this workstation" on that workstation's own row. Use this when a workstation with no printer of its own should route somewhere other than the business-wide default (e.g. WS-3/WS-4 in §11.5's scenario, routed specifically to WS-1's printer rather than whatever the business default happens to be).

11.8 Troubleshooting playbook

Symptom	Likely cause	Fix
A printer shows "Offline" on a remote workstation but prints fine locally	The printer's owning agent isn't actually connected right now (agent process not running, machine off, network down) — this is a real, live status, not a caching bug.	Check the workstation's tray icon / Admin → Workstation Agents connection status for that pairing. If it says Connected but the printer still shows offline, use Check Status / Bring Online to force a fresh check.
Same printer name appears twice in the picker or admin list, one always offline	A stale "zombie" NetworkPrinters row left over from a revoked pairing that was later re-paired (re-pairing always creates a new WorkstationAgents row, never reuses the old one).	Should self-heal automatically — every listing excludes a printer whose owning agent is revoked. If you still see it, confirm you're on a build that includes this fix; it will not reappear after a fresh page load.
Setting up Business B's printer on a shared workstation made Business A's printer disappear	This was a real historical bug (pre-MBM-283): only one business's connection could be "active" per physical machine at a time.	Fixed — every paired business's connection now stays up simultaneously, regardless of which business currently has UI focus. If this recurs, it's a regression, not expected behavior — check the agent's own version against the "Agent update required" banner.
The wrong printer gets auto-selected at print time (e.g. an unreachable QZ Tray printer instead of a known-online one)	Either a stale per-workstation override pointing at a dead/disabled printer, or a "last selected" browser preference from a different business/earlier testing leaking in.	Confirm the per-workstation and business-wide defaults (§11.7c) point at a real, online printer. If the picker still won't pick the online one, it's most likely the browser's own remembered choice for <i>this specific business</i> — reselect the correct printer once from the dropdown to overwrite it.
Printer shows "Shared" but a remote workstation still can't reach it	isShareable and remoteEnabled are two different, unrelated flags — the "Shared" badge on the general admin printer list reflects the older isShareable , not whether AGENT-mode remote routing is actually on.	Check Enable remote printing and Share this printer specifically on Admin → Workstation Agents for that printer, not the isShareable badge elsewhere.
Scale readings jump to the wrong business, or a business can't connect its scale	Scale ownership is exclusive per physical machine (one serial port), unlike printers. Whichever business most recently had browser/tray focus on that machine owns it.	Use the tray's Release action (or the equivalent admin control) to hand the scale back explicitly, then refocus the correct business's tab.
"List Printers" on the setup page returns nothing	The agent must be online for this to work — it runs Get-Printer locally on that exact workstation.	Confirm the agent's connection status is green first. If it's online and still returns nothing, type the printer name manually from Windows' own Printers & Scanners settings instead — this always works regardless of whether listing does.

11.9 Appendix — relevant code

Concern	File
Printer listing / business-scoping / revoked-agent exclusion	src/lib/printing/printer-service.ts
Print-time authorization (is this printer usable by this business right now)	src/lib/printing/print-dispatch.ts
Default-printer resolution priority	src/components/receipts/unified-receipt-preview-modal.tsx
Business-wide / per-workstation default storage	src/app/api/printing/default-printer/route.ts
Pairing, printer declaration, revoke (with cleanup)	src/app/api/admin/workstation-agents/**
Local agent — multi-business persistent connections	agent/r710-local-agent/src/index.ts
Local agent — scale ownership arbitration	agent/r710-local-agent/src/scale-owner.ts

12. Backup

```
npm run backup:database
```

wraps `scripts/backup-database.js`. There isn't a separate, dedicated backup/restore reference document in this repo at the time of writing — if you need more detail than the script itself provides, read `scripts/backup-database.js` directly.

13. Post-deployment checklist

Server:

- ☐ `ENCRYPTION_KEY` set (64 hex chars) — §4.2
- ☐ `NEXTAUTH_SECRET` set to a real generated value, not the placeholder in `.env.example`
- ☐ Logged in as `admin@business.local` and **changed the default password**
- ☐ `npm run db:deploy` completed with no errors (453+ migrations applied)
- ☐ `npm run setup:production -- --no-admin --ignore-missing-models` completed (reference data seeded)
- ☐ `npm run build` completed, `dist/server.js` and `.next/BUILD_ID` both exist
- ☐ Decided TLS vs HTTP (§7.1) and, if TLS, confirmed the `HTTPS enabled` log line
- ☐ If using QZ Tray anywhere: generated the server's own QZ signing certificate (§7.2) rather than relying on QZ's demo cert long-term
- ☐ Chosen and installed one of the three run options in §8 — for real production, that's Option C
- ☐ Confirmed the PostgreSQL Windows service name matches what `windows-service/config.js` expects, if using Option C
- ☐ `CORS_ORIGINS` covers however this server will actually be reached
- ☐ A backup (`npm run backup:database`) has been taken at least once and you know where it writes to

Each workstation:

- ☐ Browser can reach the server and (if TLS) trusts its `rootCA.pem` (§10.1)
 - ☐ QZ Tray installed + the server's `qz-certificate.pem` added via Site Manager, for any workstation printing that way (§10.2)
 - ☐ Printer driver installed and visible in Windows Devices & Printers, for any printer that isn't QZ/agent-relayed (§10.4)
 - ☐ R710/Workstation local agent downloaded, running, and paired — for remote R710 devices or a workstation-attached scale/printer (§10.3), with "Start with Windows" enabled — see §11 for the full architecture and a troubleshooting playbook
 - ☐ Electron kiosk set up, only if that workstation drives a customer-facing second monitor (§10.5)
-

14. Known rough edges (worth knowing about, not necessarily worth fixing before you deploy)

- `scripts/install/install.js`'s Node version check says "16+" — ignore it; the real requirement (via Next.js 15) is Node ≥ 18.18 . Use Node 20 LTS.
- If `scripts/create-admin.js` is ever deleted or renamed, `install-database.js` falls back to a `createBasicAdmin()` path that hashes the password with plain SHA-256 (not bcrypt) and writes to a field name that doesn't match the current Prisma schema. This fallback is not currently reachable in a normal deployment — just don't delete `scripts/create-admin.js`.
- `.env.example` is missing `ENCRYPTION_KEY` and `QZ_PRIVATE_KEY/QZ_CERTIFICATE` entirely (§4.2) — don't assume it's a complete list of what the app needs.